

Vertex Boot Manager Specification

Version: 1.0

Status: Draft

1. Objetivo

O Boot Manager é o subsistema do Vertex Core responsável por controlar todo o processo de inicialização da plataforma após o Core ser iniciado pelo Seed.

Seu objetivo é garantir que todos os componentes do Core e os plugins sejam carregados na ordem correta, respeitando dependências, políticas de segurança e disponibilidade de recursos.

O Boot Manager não inicia o Android nem o Seed; ele inicia exclusivamente o ambiente de execução do Vertex.

2. Responsabilidades

O Boot Manager é responsável por:

- iniciar os subsistemas do Core;
- verificar dependências entre serviços;
- controlar a sequência de inicialização;
- inicializar plugins de sistema;
- inicializar plugins do usuário;
- interromper a inicialização em caso de falhas críticas;
- acionar mecanismos de recuperação quando necessário;
- registrar métricas e eventos de boot.

3. Arquitetura

O Boot Manager é dividido em componentes especializados.

Boot Manager

- ├── Boot Coordinator
- ├── Dependency Resolver
- ├── Service Loader
- ├── Plugin Loader
- ├── Boot Validator
- ├── Boot Recovery
- └── Boot Metrics

└─ Boot State Controller

Cada componente possui responsabilidade única.

4. Componentes

Boot Coordinator

Coordena toda a sequência de inicialização.

Dependency Resolver

Resolve dependências entre os subsistemas do Core e entre plugins de sistema.

Service Loader

Inicializa os serviços internos do Core.

Exemplos:

- Scheduler;
- Resource Manager;
- Security Framework;
- State Engine;
- Event System.

Plugin Loader

Carrega os plugins conforme sua prioridade de inicialização.

Primeiro:

- plugins obrigatórios do sistema.

Depois:

- plugins opcionais.

Por fim:

- plugins do usuário.

Boot Validator

Verifica:

- consistência;
- dependências;
- versões;
- estado dos serviços.

Boot Recovery

Executa ações de recuperação quando um serviço crítico não pode ser inicializado.

Boot Metrics

Coleta métricas do processo de inicialização.

Boot State Controller

Gerencia os estados do Boot Manager.

5. Fluxos

Inicialização Normal

Seed

↓

Core

↓

Boot Manager

↓

Serviços do Core



Plugins de Sistema



Plugins do Usuário



Sistema Pronto

Inicialização com Falha

Boot



Erro



Recovery



Nova tentativa



Sucesso

ou

Falha crítica

Inicialização Parcial

Caso um plugin opcional falhe:

Plugin

↓

Erro

↓

Registro

↓

Continuação da inicialização

O sistema permanece utilizável.

6. APIs

Boot API

- start()
- stop()
- restart()

Service API

- registerService()
- startService()
- stopService()
- isServiceRunning()

Plugin API

- loadSystemPlugins()
- loadUserPlugins()
- unloadPlugin()

Status API

- currentState()
- bootProgress()

- bootReport()

7. Estados

Idle

↓

Starting

↓

Loading Core Services

↓

Loading System Plugins

↓

Loading User Plugins

↓

Running

Estados adicionais:

- Recovering;
- Restarting;
- Failed;
- Shutdown.

8. Políticas

Ordem determinística

A sequência de inicialização deve ser sempre previsível.

Dependências

Nenhum componente pode iniciar antes de suas dependências estarem disponíveis.

Recuperação

Falhas recuperáveis devem ser tratadas automaticamente.

Continuidade

Falhas em componentes opcionais não devem impedir a inicialização completa da plataforma.

Segurança

Plugins não são iniciados antes que o Security Framework esteja operacional.

9. Integrações

O Boot Manager integra-se com:

- Plugin Manager;
- Scheduler;
- Resource Manager;
- Security Framework;
- Logging Framework;
- Configuration System;
- Android Bridge;
- Update System;
- Power Management;
- System Event Core.

O Boot Manager não acessa diretamente plugins.

Toda interação ocorre através do Plugin Manager.

10. Métricas

O Boot Manager registra:

- tempo total de boot;

- tempo por subsistema;
- tempo de carregamento de plugins;
- quantidade de serviços iniciados;
- quantidade de plugins carregados;
- falhas durante o boot;
- tentativas de recuperação;
- reinicializações necessárias.

Essas métricas podem ser utilizadas pelo Plugin Inspector e pelo Logging Framework.

11. Exemplos

Boot Normal

1. Seed inicia o Core.
2. Boot Manager inicia.
3. Security Framework é carregado.
4. Scheduler é iniciado.
5. Resource Manager é iniciado.
6. Event System é iniciado.
7. State Engine é iniciado.
8. Plugin Manager é iniciado.
9. Plugins de sistema são carregados.
10. Plugins do usuário são carregados.
11. Plataforma pronta.

Plugin Obrigatório Falha

1. Plugin obrigatório falha.
2. Boot Recovery tenta reinicializar.
3. Persistindo a falha, o Boot Manager encerra a inicialização.
4. O Seed poderá iniciar o processo de recuperação do Core.

Plugin Opcional Falha

1. Plugin opcional falha.
2. O erro é registrado.
3. O plugin permanece desativado.
4. O restante da plataforma continua iniciando normalmente.

12. Regras Arquiteturais

- O Boot Manager não executa lógica de plugins.
- O Boot Manager não implementa serviços do Core.
- Toda inicialização deve passar pelo Dependency Resolver.
- Nenhum subsistema deve depender da ordem de carregamento além das dependências declaradas.
- O processo de boot deve ser idempotente sempre que possível, permitindo reinicializações seguras.

13. Limitações

O Boot Manager não é responsável por:

- instalar plugins;
- atualizar plugins;
- atualizar o Core;
- controlar permissões;
- executar tarefas periódicas;
- gerenciar interfaces de usuário.

Essas responsabilidades pertencem a outros subsistemas.

14. Futuras Extensões

O Boot Manager poderá evoluir para suportar:

- inicialização paralela de serviços independentes;
- carregamento progressivo (lazy boot) de plugins não essenciais;
- perfis de inicialização (rápido, completo ou econômico);
- análise automática de gargalos no processo de boot;
- otimização baseada no histórico de uso do dispositivo.

Essas evoluções devem manter a previsibilidade e a confiabilidade da inicialização.

15. Resumo

O Boot Manager coordena toda a inicialização do Vertex Core, garantindo que serviços e plugins sejam carregados de forma segura, previsível e eficiente. Ele respeita dependências, isola falhas, coleta métricas e prepara a plataforma para operação, constituindo um dos principais responsáveis pela estabilidade e pelo tempo de inicialização do Vertex.